

Programming SQL Server 2016

Stored Procedures

October 23, 2019 9:43:31 AM

Prelude

You have already learnt about VIEWS. In this lesson, we are going to learn another very important object – Stored Procedure. As a developer you will frequently use Stored Procedures for various purposes.

Go ahead and get started.

Today's topics

- Store Procedure – What is it?
- How to create stored procedures
- How to execute Store Procedures
- Returning a value from stored procedures
- Procedure parameter with default values
- INPUT and OUTPUT parameters
- Returning values from procedures using OUTPUT parameters
- Applying procedural integrity constraints using stored procedures
- Workouts
- Practice works

Home works

- A. Write a simple stored procedure that returns the desired Customer record from the Northwind database given a parameter of the CustomerID.
- B. Write a stored procedure that accepts a Territory ID, Territory Description, and Region ID and inserts them as new row in the Territories table in Northwind.
- C. Alter the procedure you created in #2 to pre-check for the existence of the foreign key (RegionID) before attempting the insert. If the RegionID doesn't exist, throw an error with the error text "RegionID is not valid. Please check your RegionID and try again".

Important Points:

-
-
-
-

Summary:

-
-
-
-

What is a stored procedure?

- A stored procedure (AKA procedure or SPROC) is a set of SQL statements stored as an object and which is treated as a unit
- Whenever a stored procedure is executed, all statements within it are executed
- A Stored procedure can accept parameter to pass data during execution and can return value.

What can we do with a SPROC?

- The use of stored procedures is diverse.
- A stored procedure can hold any type and number of T-SQL statements [There are a few of exceptions].

Advantages of Stored Procedures

- Usually better performance
- Possible use as a security insulation layer (control how a database is accessed and updated)
- Reusable code
- Compartmentalization of code (can encapsulate business logic)
- Flexible execution depending on dynamics established at runtime.

Basic Syntax

- In simple form
CREATE PROCEDURE procedure-name param1, param2, ...
AS
SQL statement
...
GO
- With option
CREATE PROCEDURE procedure-name param1, param2, ...
WITH RECOMPILE, ENCRYPTION
AS
SQL statement
...
GO

Procedures may not have parameters

Running a stored procedure

```
EXECUTE procedure-name param1, param2,..  
OR  
EXECUTE procedure-name param1=value1, param2=value2,...
```

EXEC is short form of EXECUTE

i While executing a stored procedure, placing EXEC/EXECUTE keyword before the procedure name is optional if the statement is the first one in the batch. Otherwise you must use the EXEC/EXECUTE

Workouts

- Let's play with some stored procedures
- Let's create a table and insert some sample data into the table
- Create the table
CREATE TABLE products
(
productid int identity primary key,
productname nvarchar(40) not null,
category nvarchar(40) not null,
unitprice money null,
available bit default 0
)
GO
- Insert sample data
INSERT INTO products
(productname, category, unitprice, available)

VALUES

```
('HDD 500GB', 'Storage Device', 5700.00, 1)  
( 'HDD 640GB', 'Storage Device', 6300.00, 0),  
( 'SSD 120GB', 'Portable Storage', 10500.00,1),  
( 'A4 Tech Mouse', 'Accessories', 550.00, 1),  
( 'Wireless Keyboard', 'Accessories', 900.00,0)  
GO
```

- Check data
SELECT * FROM products
GO
- Result

productid	productname	category	unitprice	available
1	HDD 500GB	Storage Device	5700.00	1
2	HDD 640GB	Storage Device	6300.00	0
3	SSD 120GB	Portable Storage	10500.00	1
4	A4 Tech Mouse	Accessories	550.00	1
5	Wireless Keyboard	Accessories	900.00	0

Workout #01

- Let's create a simple procedure that retrieves all the products from products table
CREATE PROC spProductsAll
AS
SELECT productname, category, unitprice, available
FROM products
GO
- Test the procedure
EXEC spProductsAll
GO
- Result

productname	category	unitprice	available
HDD 500GB	Storage Device	5700.00	1
HDD 640GB	Storage Device	6300.00	0
SSD 120GB	Portable Storage	10500.00	1
A4 Tech Mouse	Accessories	550.00	1
Wireless Keyboard	Accessories	900.00	0

Workout #02

- Though the procedure we have just created perform better than the plain select statement, the producedure is not that much useful.
- Let's create another procedure
CREATE PROC spProductsAvailable
AS
SELECT productname, category, unitprice, available
FROM products WHERE available = 1
GO

i PROC is the short form of PROCEDURE
Always use GO to terminate a procedure definition

- Test the procedure
EXEC spProductsAvailable
GO
- Result

productname	category	unitprice	available
HDD 500GB	Storage Device	5700.00	1
SSD 120GB	Portable Storage	10500.00	1
A4 Tech Mouse	Accessories	550.00	1

- This procedure shows only available products

Input parameters in stored procedure

- Parameters are by default input parameter
- Input parameters are used to pass data to procedure during procedure call
- Syntax of declaring parameter is
@parameter-name data-type
- You can declare parameter with default value, syntax is
@parameter-name data-type = default-value

Workout #03

- Let's create a procedure with parameter
- In simple form parameter
CREATE PROC spProductsOfCategory @category nvarchar (40)
AS
SELECT productname, category, unitprice, available
FROM products WHERE category = @category
GO
- Test the procedure
EXEC spProductsOfCategory 'Storage Device'
GO
- Result

productname	category	unitprice	available
HDD 500GB	Storage Device	5700.00	1
HDD 640GB	Storage Device	6300.00	0

- You can pass parameter value using parameter name like below
EXEC spProductsOfCategory @category =
'Portable Storage'
GO
- Result

productname	category	unitprice	available
SSD 120GB	Portable Storage	10500.00	1

Workout #04

- Let's create another procedure.
- The procedure is to insert data into the products table.
- The procedure requires four parameters to pass data for productname, category unitprice and available fields.
- Be careful data types for the parameters, they must match and long enough with column data types so that no conversion fails and no value truncates
CREATE PROC spProductInsert
@productname nvarchar(40),
@category nvarchar(40),
@price money, @available bit
AS
INSERT INTO products
(productname, category, unitprice, available)
VALUES (@productname, @category,@price, @available)
GO
- Test the procedure
EXEC spProductInsert 'Multimedia Keyboard',
'Accessories', 350.00, 1
GO
EXEC spProductInsert @productname='Transcend 500GB',
@category = 'Portable Storage', @price=6300.00,
@available=0
GO
- Check the data
SELECT * FROM products
GO

i

When a procedure parameter has no default value the parameter becomes required, you must pass value for it. If you pass values for parameters without mentioning their names you must maintain the order.

But if you pass values for parameters mentioning their names, the order of parameters is not important. The above procedure can be executed like below:
EXEC spProductInsert @category = 'Portable Storage',@productname='Transcend 500GB',@price=6300.00, @available=0

productid	productname	category	unitprice	available
1	HDD 500GB	Storage Device	5700.00	1
2	HDD 640GB	Storage Device	6300.00	0
3	SSD 120GB	Portable Storage	10500.00	1
4	A4 Tech Mouse	Accessories	550.00	1
5	Wireless Keyboard	Accessories	900.00	0
6	Multimedia Keyboard	Accessories	3500.00	1
7	Transcend 500GB	Portable Storage	6300.00	0

Returning value from a stored procedure

- A stored procedure can return only one value
- In T-SQL, a procedure can return only integer value
- Use RETURN keyword to return a value
- To capture the return value from a procedure, you must use a variable which is, of course, integer type.
- Syntax is

EXEC @variable = procedure-name parameters

- You must declare the variable before using it

Workout #05

- Let's create another procedure
- The procedure is to insert data into the products table like workout #04. But this one returns a value.
- The return value is the productid generated by SQL Server for the newly inserted product. This is identity value.
CREATE PROC spProductInsertWithReturn
@productname nvarchar(40), @category nvarchar(40),
@price money, @available bit
AS
DECLARE @productid int
INSERT INTO products (productname, category, unitprice, available) VALUES (@productname, @category,@price, @available)
SELECT @productid = IDENT_CURRENT('products')
RETURN @productid
GO
- Test the procedure
DECLARE @id int
EXEC @id = spProductInsert 'Pen Drive 8GB',
'Removable Storage', 600.00, 1
PRINT 'New product inserted with id: ' + str(@id)
GO
- Output
New product inserted with id: 8

- See the data
SELECT * FROM products
GO

Result

productid	productname	category	unitprice	available
1	HDD 500GB	Storage Device	5700.00	1
2	HDD 640GB	Storage Device	6300.00	0
3	SSD 120GB	Portable Storage	10500.00	1
4	A4 Tech Mouse	Accessories	550.00	1
5	Wireless Keyboard	Accessories	900.00	0
6	Multimedia Keyboard	Accessories	3500.00	1
7	Transcend 500GB	Portable Storage	6300.00	0
8	Pen Drive 8GB	Removable Storage	600.00	1

Using default value of parameters

- A parameter of a stored procedure can have default value
- The default value is used when you do not give any value for the parameter
- Default value makes a parameter optional
- Basic syntax for declaring a parameter

@param-name data-type=default value

Workout #06

- Let's try a procedure with parameters default values.
- We will create another procedure with parameter default values to insert product record.

Create the procedure

```
CREATE PROC spProductInsertWithDefaultValues
@productname NVARCHAR(40) = NULL,
@category NVARCHAR(40) = 'Misc. Parts',
@price MONEY = NULL,
@available BIT = 0
AS
INSERT INTO products
(productname, category, unitprice, available)
VALUES (@productname, @category, @price, @available)
GO
```

Test the procedure

```
EXEC spProductInsertWithDefaultValues
@productname = 'Laptop Cooler', @price = 1100.00
GO
SELECT * FROM products
GO
```

productid	productname	category	unitprice	available
1	HDD 500GB	Storage Device	5700.00	1
2	HDD 640GB	Storage Device	6300.00	0
3	SSD 120GB	Portable Storage	10500.00	1
4	A4 Tech Mouse	Accessories	550.00	1
5	Wireless Keyboard	Accessories	900.00	0
6	Multimedia Keyboard	Accessories	3500.00	1
7	Transcend 500GB	Portable Storage	6300.00	0
8	Pen Drive 8GB	Removable Storage	600.00	1
9	Laptop Cooler	Misc. Parts	1100.00	0

- Look default values for category and available columns are used since these two values are not provided to the procedure

OUTPUT parameters

- A parameter can have direction
- By default all parameters are INPUT parameters, that is, are in INPUT direction
- Input parameters are used to pass values to a stored procedure when running the stored procedure
- Changes in input parameters are not visible after execution of the procedure
- Stored procedures can have OUTPUT parameters
- Any change in output parameters are visible after execution of the procedure
- Since stored procedure can only return integer value, OUTPUT parameters are useful to return values of other data types
- Since stored procedures can return only one value, output parameters are used to return multiple values
- Basic syntax of declaring OUTPUT parameter:
@param-name data-type OUTPUT
- Or
@param-name data-type = default-value OUTPUT

Passing output parameter

- Variables of appropriate types are passed as output parameters which are declared before passing
- When passing a variable as output parameter to a procedure it must be marked as output
- After running the procedure you can get the value set in the procedure from the variable
- The following workout will clear the things

Workout #07

- Let's create another procedure
- This procedure also insert data into products table but accepts a output parameter
- This procedure sets the productid assigned to the inserted product

Create the procedure

```
CREATE PROC spProductInsertWithOutParam
@productname NVARCHAR (40),
@category NVARCHAR (40),
@price MONEY, @available BIT,
@id INT OUTPUT
AS
INSERT INTO products
(productname, category, unitprice, available)
VALUES (@productname, @category, @price, @available)
SELECT @id = IDENT_CURRENT('products')
GO
```

Test the procedure

```
DECLARE @productid int
EXEC spProductInsertWithOutParam 'DDR3 4GB',
'Memory Module', 1350.00, 1, @productid OUTPUT
--Do not place go here. If you place go here @productid
--variable will get out of scope
SELECT @productid 'New Id'
GO
```

New Id

10

- See table data
SELECT * FROM products
GO

productid	productname	category	unitprice	available
1	HDD 500GB	Storage Device	5700.00	1
2	HDD 640GB	Storage Device	6300.00	0
3	SSD 120GB	Portable Storage	10500.00	1
4	A4 Tech Mouse	Accessories	550.00	1
5	Wireless Keyboard	Accessories	900.00	0
6	Multimedia Keyboard	Accessories	3500.00	1
7	Transcend 500GB	Portable Storage	6300.00	0
8	Pen Drive 8GB	Removable Storage	600.00	1
9	Laptop Cooler	Misc. Parts	1100.00	0
10	DDR3 4GB	Memory Module	1350.00	1

Applying procedural integrity

- Stored procedure can be used to apply integrity constraint in procedural way
- For example suppose you did not specify required check constraint on a column value, you can use stored procedure in this case
- Instead of inserting data directly, you can use a procedure and check value for that column
- Even you can apply foreign key constraint using stored procedure

Workout #08

- Let's create another procedure to apply procedural constraint
- We want to ensure that the price column value be non-zero positive value
- We will use a stored procedure to check price column value while inserting and reject any zero or negative value
- Create the procedure

```
CREATE PROC spProductInsertWithCheckPrice
@productname NVARCHAR(40), @category NVARCHAR(40),
@price MONEY, @available BIT
AS
IF @price > 0
    INSERT INTO products
    (productname, category, unitprice, available )
    VALUES ( @productname, @category, @price, @available)
ELSE
BEGIN
    RAISERROR ('Price cannot be 0 or -ve', 10, 1)
    RETURN
END
GO
```
- Test the procedure

```
EXEC spProductInsertWithCheckPrice 'Bluetooth Adpter',
'Accessories', 0, 1
GO
```
- You will see an error message
- Price cannot be 0 or -ve
- But the following will work

```
EXEC spProductInsertWithCheckPrice 'Bluetooth Adpter',
'Accessories', 250.00, 1
GO
```
- See table data

```
SELECT * FROM products
GO
```

productid	productname	category	unitprice	available
-----------	-------------	----------	-----------	-----------

1	HDD 500GB	Storage Device	5700.00	1
...	...	..	...	...
11	Bluetooth Adapter	Accessories	350.00	1

Practice #01

- A. Create a table Reservations as defined

i

RAISERROR is a built-in function to report error and show error message.

The function is called like below:
RAISERROR (message, severity, state)

This function has other forms to call.

We will explore this function in more detail in later session.

Column	Data type	Nullability
Id	IDENTITY	NOT NULL
Room	INT	NOT NULL
Date	DATE	NOT NULL

- B. Create a stored procedure with appropriate parameters to insert data into the Reservations table.
- C. Insert the following data into the table using the procedure you created

Id	Room	Date
1	105	2012-01-01
2	109	2012-02-21
3	114	2012-02-01

- D. Create a procedure to update data in Reservations table.
- E. Update the record as shown below using the procedure created in D

Id	Room	Date
2	115	2012-02-21

- F. Create a procedure that accepts month and year and selects the reservation records in the supplied month of the supplied year.

Practice #02

- A. Create the "Promotion Sales" as described below

Column	Datatype	Nullability
Product	VARCHAR(30)	NOT NULL
Price	MONEY	NOT NULL
DiscountRate	INT	NOT NULL

- B. Create a stored procedure to insert data into the "Promotion Sales" table. Define necessary parameter and make sure the parameter for DiscountRate value has default value 5 so that if DiscountRate is not supplied, the default value is inserted.
- C. Insert the following records into "Promotion Sales" using the procedure

Product	Price	DiscountRate
Jeans	1050	12
Shoe	1200	7
Shirt	800	5

- D. Create a procedure that select all the column in "Promotion Sales" and an extra column that shows

discounted price by deducting discount from price. Its result will be like below

Product	Price	DiscountRate	Sale Price
Jeans	1050.00	12	924.00
Shoe	1200.00	7	1116.00
Shirt	800.00	5	760.00

Practice #03

A. Create a table "Classes" as Described

Column	Data type	Nullability
ClassId	INT	NOT NULL
Description	VARCHAR (100)	NOT NULL
ClassHour	INT	NOT NULL

- B. You want to apply procedural integrity on column CourseHour instead of check constraint. Create procedure to insert data into "Classes" table and make sure that value in ClassHour column is not less than 2.
- C. Insert some test values and show that the integrity rule is working

Practice #04

A. Create a table learningmaterial as described:

Column	Data type	Nullability
id	INT	NOT NULL
subject	VARCHAR(100)	NOT NULL
format	VARCHAR(20)	NOT NULL
price	MONEY	NOT NULL

- B. Create a stored procedure to insert data into the table. The procedure checks the value given for the format field, it accepts the value if it matches any of the values - "CD/DVD", "PDF", "MLEARNING", otherwise it shows error message "Invalid format". The procedure changes the value for the format column into uppercase before inserting.
- C. Store the following rows using the procedure you created in B

id	subject	format	price
1	MS Excel functions	CD/DVD	12.50
2	PIVOT tables in MS Access	PDF	07.50
3	HTML5 and CSS3	PDF	19.33
4	Flash CS6	CD/DVD	17.99
5	Photoshop CS6	CD/DVD	25.99
6	Objective C	PDF	09.99

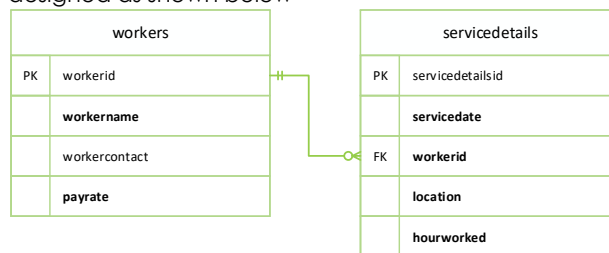
- D. Create a procedure that accepts a parameter of type VARCHAR (20). The procedure select records from learningmaterial tables by matching the format field value with the value supplied as parameter.

Practice #05

- A. Create a procedure that accepts country name as parameter and shows all the customers living in the supplied country from Customers table in Northwind sample database
- B. Create a procedure that takes two DateTime type parameter and shows order placed within the supplied dates from Orders table in Northwind sample database.

Practice #06

A home service provider company has some workers to serve customers at home. The Company wants to use a database to store service details. Two tables are designed as shown below



* **bold faced attributes are required**

- A. Create the two tables with relation.
- B. Create a procedure to insert data into workers. Use appropriate parameters and default values.
- C. Create a procedure to insert data into servicedetails. Use appropriate parameters and default values.
- D. Insert the following rows in workers table using the procedure you created

workerid	workername	workercontact	payrate
1	Hard Worker	019999999	300.00
2	Jack of all trades		275.00
3	Lazy Joe	019999999	250.00
4	Sleeping tom		200.00

- E. Insert the following in servicedetails using the procedure you just created

The day before Yesterday the flowing service has been provided

workerid	location	hourworked
1	Dhanmondi	3.5
4	Mirpur	2.5
2	Firmgate	1.0
2	Firmgate	4.0
1	Mirpur	6.5

Yesterday the flowing service has been provided

workerid	location	hourworked
2	Dhanmondi	3.5
1	Mirpur	2.5
2	Firmgate	1.0
3	Firmgate	4.0
4	Mirpur	6.5

- F. Create a view that shows workerid, workername along with total hourworked, total pay in the current month.
[Pay is calculated using formula payrate*hourworked]
- G. Create a view that shows workerid, workername along with total hourworked, total pay in the last 7 days.
- H. Create a view that shows workerid, workername along with his services including servicedate, location and hourworked.

Practice #07

You have two tables like below

Department	
PK	departmentid
	departmentname

Employee	
PK	employeeid
	employeename
	joindate
	departmentid

You did not apply referential integrity rules during table creation. You want to enforce it in procedural way.

- A. Create a procedure to insert employee and check foreign key requirement on departmentid field in the procedure.
- B. Create a procedure to delete a department and make sure that a department is not deleted if any employee exists in the department.